

Практическая работа 6

Разработка помощника принятия решения

Для начала посмотрим код:

```
import flet as ft

def main(page: ft.Page):
    page.title = 'Дерево решений'
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    questions = {
        'root': {'text': 'Запускать новый продукт?', 'yes': 'high_demand', 'no':
'low_demand'},
        'high_demand': {'text': 'Высокий спрос?', 'yes': 'high_profit', 'no': 'low_profit'},
        'high_profit': {'text': 'Высокая прибыль?', 'yes': 'low_competition', 'no':
'high_competition'},
        'low_competition': {'text': 'Низкая конкуренция?', 'yes':
'result_launch_immediately', 'no': 'result_utp'},
        'high_competition': {'text': 'Высокая конкуренция?', 'yes':
'result_analyze_competitors', 'no': 'result_find_niche'},
        'low_profit': {'text': 'Низкая прибыль?', 'yes': 'low_cost', 'no': 'high_cost'},
        'low_cost': {'text': 'Низкая себестоимость?', 'yes': 'result_launch_cautiously',
'no': 'result_optimize_production'},
        'high_cost': {'text': 'Высокая себестоимость?', 'yes': 'result_find_investors', 'no':
'result_review_business_plan'},
        'low_demand': {'text': 'Есть сезонность?', 'yes': 'peak_soon', 'no':
'peak_not_soon'},
        'peak_soon': {'text': 'Пик спроса скоро?', 'yes': 'result_prepare_launch', 'no':
'result_delay_launch'},
        'peak_not_soon': {'text': 'Пик спроса не скоро?', 'yes':
'result_develop_low_season', 'no': 'result_find_new_market'},
    }

    results = {
        'result_launch_immediately': 'Запускать немедленно',
        'result_utp': 'Нужно УТП',
        'result_analyze_competitors': 'Анализировать конкурентов',
        'result_find_niche': 'Искать нишу',
        'result_launch_cautiously': 'Запускать с осторожностью',
        'result_optimize_production': 'Оптимизировать производство',
        'result_find_investors': 'Искать инвесторов',
        'result_review_business_plan': 'Пересмотреть бизнес-план',
        'result_prepare_launch': 'Готовиться к запуску',
        'result_delay_launch': 'Отложить запуск'
```

```

    'result_develop_low_season': ' Развивать в низкий сезон',
    'result_find_new_market': ' Искать новый рынок',
}

```

```

current_key = 'root'
history_keys = []
history_answers = []

```

```

question_text = ft.Text('Запускать новый продукт?', size=24,
weight=ft.FontWeight.BOLD)

```

```

result_text = ft.Text("", size=20)

```

```

history_text = ft.Text('История: пусто', size=14)

```

```

tree_view = ft.ListView(expand=True)

```

```

def build_tree():

```

```

    tree_view.controls.clear()

```

```

    tree_view.controls.append(ft.Text('📁 Запуск нового продукта?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Высокий спрос?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Высокая прибыль?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Низкая конкуренция?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Запускать немедленно', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Нужно УТП', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Высокая конкуренция?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Анализировать конкурентов',
size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Искать нишу', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Низкая прибыль?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Низкая себестоимость?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Запускать с осторожностью',
size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Оптимизировать производство',
size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Высокая себестоимость?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Искать инвесторов', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Пересмотреть бизнес-план',
size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Низкий спрос?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Есть сезонность?', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Пик спроса скоро?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Готовиться к запуску', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Отложить запуск', size=14))

```

```

    tree_view.controls.append(ft.Text('📁 Пик спроса не скоро?', size=14))

```

```

    tree_view.controls.append(ft.Text('📄 Развивать в низкий сезон',
size=14))

```

```
tree_view.controls.append(ft.Text('Искать новый рынок', size=14))
page.update()
```

```
def on_yes(e):
```

```
    nonlocal current_key, history_keys, history_answers
```

```
    if current_key in questions:
```

```
        q = questions[current_key]
```

```
        if 'yes' in q:
```

```
            history_keys.append(current_key)
```

```
            history_answers.append('Да')
```

```
            next_key = q['yes']
```

```
            if next_key.startswith('result_'):
```

```
                result_text.value = 'Результат: ' + results[next_key]
```

```
                result_text.color = 'green'
```

```
                history_text.value = 'История: ' + ' -> '.join(history_answers) + ' -> ' +
```

```
results[next_key]
```

```
                current_key = next_key
```

```
            else:
```

```
                current_key = next_key
```

```
                question_text.value = questions[current_key]['text']
```

```
                history_text.value = 'История: ' + ' -> '.join(history_answers)
```

```
    page.update()
```

```
def on_no(e):
```

```
    nonlocal current_key, history_keys, history_answers
```

```
    if current_key in questions:
```

```
        q = questions[current_key]
```

```
        if 'no' in q:
```

```
            history_keys.append(current_key)
```

```
            history_answers.append('Нет')
```

```
            next_key = q['no']
```

```
            if next_key.startswith('result_'):
```

```
                result_text.value = 'Результат: ' + results[next_key]
```

```
                result_text.color = 'red'
```

```
                history_text.value = 'История: ' + ' -> '.join(history_answers) + ' -> ' +
```

```
results[next_key]
```

```
                current_key = next_key
```

```
            else:
```

```
                current_key = next_key
```

```
                question_text.value = questions[current_key]['text']
```

```
                history_text.value = 'История: ' + ' -> '.join(history_answers)
```

```
    page.update()
```

```
def on_back(e):
```

```
    nonlocal current_key, history_keys, history_answers
```

```

if history_keys:
    current_key = history_keys.pop()
    if history_answers:
        history_answers.pop()
    if current_key in questions:
        question_text.value = questions[current_key]['text']
        result_text.value = "
        history_text.value = 'История: ' + ' -> '.join(history_answers) if
history_answers else 'История: пусто'
        page.update()

def on_restart(e):
    nonlocal current_key, history_keys, history_answers
    current_key = 'root'
    history_keys = []
    history_answers = []
    question_text.value = 'Запускать новый продукт?'
    result_text.value = "
    history_text.value = 'История: пусто'
    page.update()

build_tree()

left_panel = ft.Container(content=ft.Column([ft.Text('Дерево', size=18,
weight=ft.FontWeight.BOLD), tree_view]), width=250, padding=10)
center_panel = ft.Column([question_text, result_text, history_text,
ft.Row([ft.Button('Да', on_click=on_yes), ft.Button('Нет', on_click=on_no)]),
ft.Row([ft.Button('Назад', on_click=on_back), ft.Button('Сброс',
on_click=on_restart)])], spacing=20)

page.add(ft.Row([left_panel, center_panel]))

if __name__ == "__main__":
    ft.app(target=main)

```

Теперь посмотрим на работу приложения на рисунке 1:

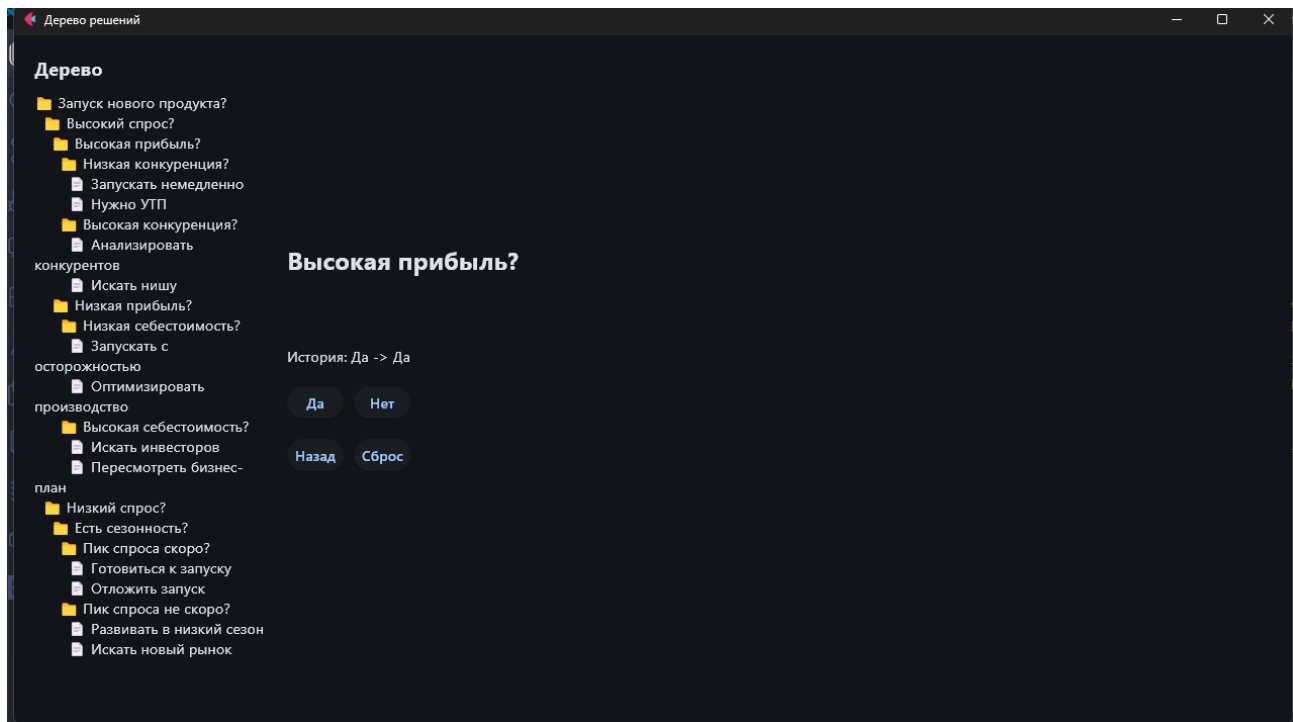


Рисунок 1 — Демонстрация работы